~\Desktop\Advanced Applied Deep Learning In Physics And Engineering\Exercises\Exercise 7 - Diffusion Generation of MNIST digists\Diffusion MNIST.py

```python
import torchvision
# For image transforms
from torchvision import transforms
# For DATA SET
import torchvision.datasets as datasets
# For Pytorch methods
import torch
import torch.nn as nn
# For Optimizer
import torch.optim as optim
# FOR DATA LOADER
from torch.utils.data import DataLoader, random_split
import os
# MODEL
from denoising_diffusion_pytorch import Unet, GaussianDiffusion
from tqdm import tqdm
# FOR PLOTTING
import matplotlib.pyplot as plt


# Hyperparameters
LEARNING_RATE = 4e-4
BATCH_SIZE = 128
N_EPOCHS = 30
IMAGE_SIZE = 28
TIME_STEPS = 1000
SAMPLING_TIMESTEPS = 250
NUM_EXAMPLES = 3

# we define a tranform that converts the image to tensor
myTransforms = transforms.Compose([transforms.ToTensor()]) # maps [0,255] → [0.0, 1.0]

# Full MNIST training set
full_train_dataset = datasets.MNIST(root="dataset/", train=True, transform=myTransforms, download=True)

# 90% training, 10% validation
train_size = int(0.9 * len(full_train_dataset))  # 54,000
val_size = len(full_train_dataset) - train_size  # 6,000
train_dataset, val_dataset = random_split(full_train_dataset, [train_size, val_size])

# Loaders
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Test set stays as it is
```

```
test_dataset = datasets.MNIST(root='dataset/', train=False, download=False, transform=myTransforms)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# ============================== Training ==============================
def train_model():
    train_losses, val_losses = [], []

    for epoch in range(N_EPOCHS):
        model.train()
        total_train_loss = 0
        for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{N_EPOCHS}"):
            images, _ = batch
            images = images.to(device)

            loss = diffusion(images) # Pass through UNET
            optim.zero_grad()
            loss.backward()
            optim.step()

            total_train_loss += loss.item()

        avg_train_loss = total_train_loss / len(train_loader)
        train_losses.append(avg_train_loss)

        # Validation
        model.eval()
        total_val_loss = 0
        with torch.no_grad():
            for batch in val_loader:
                images, _ = batch
                images = images.to(device)
                val_loss = diffusion(images)
                total_val_loss += val_loss.item()
        avg_val_loss = total_val_loss / len(val_loader)
        val_losses.append(avg_val_loss)

        print(f"Epoch {epoch+1} | Train Loss = {avg_train_loss:.4f} | Val Loss = {avg_val_loss:.4f}")

        if (epoch + 1) % 3 == 0 or epoch == N_EPOCHS - 1:
            sampled_images = diffusion.sample(batch_size=3) # Sample images
            save_images(sampled_images, epoch + 1) # Save images

    return train_losses, val_losses


def save_images(images, epoch):
    images = images.cpu().detach()

```

```
    fig, axs = plt.subplots(1, 3, figsize=(9, 3))
    for i in range(3):
        axs[i].imshow(images[i][0], cmap="gray")
        axs[i].axis("off")

    fig.suptitle(f"Generated Digits at Epoch {epoch}", fontsize=14)

    os.makedirs("Figures", exist_ok=True)
    plt.savefig(f"Figures/{epoch}.png", bbox_inches="tight")
    plt.close()

# ==================== Evaluation ========================
def compute_test_loss():
    model.eval()
    total_test_loss = 0
    with torch.no_grad():
        for batch in test_loader:
            images, _ = batch
            images = images.to(device)
            loss = diffusion(images)
            total_test_loss += loss.item()
    return total_test_loss / len(test_loader)



# ==================== Plotting ====================

def plot_losses(train_losses, val_losses, test_loss):
    plt.figure(figsize=(8, 5))
    plt.plot(train_losses, label="Train Loss")
    plt.plot(val_losses, label="Validation Loss")
    plt.axhline(y=test_loss, color='r', linestyle='--', label=f"Test Loss ({test_loss:.4f})")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Train vs Validation vs Test Loss")
    plt.legend()
    plt.grid(True)
    plt.savefig("Figures/loss_curve.png", bbox_inches="tight")
    plt.close()


# ====================== RUN CODE ============================

DIM = 32 #  base dimensionality (number of channels) for the U-Net model.
DIM_MULTS = (1, 2, 5) # UNET with layers and channel widths; L1: 32x1, L2: 32x2, L3: 32x5.
Output dimension is implicit and same as input dim (makes sense since its a diffusion model)
model = Unet(dim = DIM, dim_mults = DIM_MULTS, flash_attn = False, channels = 1) # defines
the model
# Set up diffusion model with gaussian noise
diffusion = GaussianDiffusion(model, image_size = IMAGE_SIZE, timesteps = TIME_STEPS,
sampling_timesteps = SAMPLING_TIMESTEPS)    # number of sampling timesteps (using ddim for
```

```
    faster inference [see ddim paper])
142
143 optim = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
144 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
145 print(f"Using device: {device}")
146 model.to(device)
147 diffusion.to(device)
148
149 train_losses, val_losses = train_model()
150 test_loss = compute_test_loss()
151 plot_losses(train_losses, val_losses, test_loss)
152
153
```